Data Cleaning

Table of Contents

1. Introduction & Golden Rules
2. 1. Handling Accuracy Issues
3. 2. Consistency & Schema Issues
4. 3. Handling Missing Data
5. 4. Handling Duplicates
6. 5. Handling Outliers
7. 6. Time Series – Filling Gaps
8. 7. Handling Class Imbalance
9. 8. Building Reusable Pipelines

🌟 Introduction: Data Cleaning vs. Transformation

Data cleaning and transformation are the systematic set of procedures applied to raw datasets to ensure they are accurate, consistent, and suitable for analysis.

Stage	Purpose	Examples
🧹 **Data Cleaning**	Detect & resolve quality issues. Restores data to its *intended correct state*.	Fix wrong values, fill missing data, remove duplicates
⚙️ **Data Transformation**	Reshape data for modeling	Normalization, encoding, feature engineering

🏅 The Golden Rules of Data Cleaning

#	Rule	Why It Matters
1️⃣	**Never overwrite raw data** — always work on a copy	You must be able to re-run from scratch
2️⃣	**Log every cleaning action** (what, why, how many records)	Reproducibility & audit trail
3️⃣	**Re-run validation after cleaning** to confirm resolution	Cleaning can introduce new issues
4️⃣	**Treat cleaning as a pipeline**, not a one-off script	Reusability on new data

1. 🎯 Handling Data Accuracy Issues

> **Definition:** Data does not correctly represent the real-world entity it describes.
>
> Two cleaning strategies:
>
> - **Rule-based correction** — fix the value when the correct value *can* be inferred
> - **Record rejection** — quarantine the record when correction would introduce false data

📊 Raw Customer Data (with accuracy problems):

	customer_id	name	age	phone	email	salary
0	101	alice smith	25	(555) 123-4567	alice@email.com	50000
1	102	BOB JONES	-30	555-987-6543	bob.email.com	-5000
2	103	Charlie Brown	150	555.246.8101	charlie@ok.org	75000
3	104	diana	40	+1-555-333-4444	diana@@bad.com	48000
4	105	NaN	35	5552345678	eve@valid.net	90000
5	106	EVE MILLER	28	NaN		120000
6	107	frank	200	555 999 0000	frank@test	65000

✅ Strategy 1: Rule-Based Correction

Apply domain knowledge as code. If a value violates a rule **but the correct value can be recovered**, fix it programmatically.

✔️ **When to use:**

- The error is *systematic* (e.g., all dates in wrong format)
- The correct value can be derived from other fields
- The violation is minor and a safe default exists

❌ **When NOT to use:**

- The correct value is genuinely unknown
- Making an assumption would introduce **false signal** into your model

```python
df = customers_raw.copy()  # ✅ Golden Rule #1: work on a copy
# It is a deep copy
```

```python
# ──────────────────────────────────────────────
# Rule 1: Age must be between 0 and 120
# ──────────────────────────────────────────────
# If age is NEGATIVE → flip sign (common data entry error)
df['age'] = df['age'].apply(lambda x: abs(x) if pd.notna(x) and x < 0 else x)

# If age > 120 → unrecoverable, set to NaN for later imputation
df.loc[df['age'] > 120, 'age'] = None
```

Before:

	customer_id	age
0	101	25
1	102	-30
2	103	150
3	104	40
4	105	35
5	106	28
6	107	200

After age correction:

	customer_id	age
0	101	25.0
1	102	30.0
2	103	NaN
3	104	40.0
4	105	35.0
5	106	28.0
6	107	NaN

```python
# ──────────────────────────────────────────────
# Rule 2: Standardize phone numbers → digits only
# ──────────────────────────────────────────────
# str.replace with regex=True removes ALL non-digit characters \D
df['phone'] = df['phone'].str.replace(r'\D', '', regex=True)
```

Before:

	customer_id	phone
0	101	(555) 123-4567
1	102	555-987-6543
2	103	555.246.8101
3	104	+1-555-333-4444
4	105	5552345678
5	106	NaN
6	107	555 999 0000

After phone standardization:

	customer_id	phone
0	101	5551234567
1	102	5559876543
2	103	5552468101
3	104	15553334444
4	105	5552345678
5	106	NaN
6	107	5559990000

```python
# ──────────────────────────────────────────────
# Rule 3: Strip whitespace + fix capitalization on names
# ──────────────────────────────────────────────
# str.strip()  → removes leading/trailing spaces
# str.title()  → capitalizes first letter of each word
df['name'] = df['name'].str.strip().str.title()
```

Before:

	customer_id	name
0	101	alice smith
1	102	BOB JONES
2	103	Charlie Brown
3	104	diana
4	105	NaN
5	106	EVE MILLER
6	107	frank

After name standardization:

	customer_id	name
0	101	Alice Smith
1	102	Bob Jones
2	103	Charlie Brown
3	104	Diana
4	105	NaN
5	106	Eve Miller
6	107	Frank

```python
# ──────────────────────────────────────────────
# Rule 4: Flag invalid emails (do NOT guess the correction)
# ──────────────────────────────────────────────
# Regex pattern: one or more chars, then @, then domain, then .extension
df['email_valid'] = df['email'].str.contains(
    r'^[^@]+@[^@]+\.[^@]+$', regex=True, na=False
)
# na=False ==> means put False if na (important in older pandas versions)
```

Email validation results:

	customer_id	email	email_valid
0	101	alice@email.com	True
1	102	bob.email.com	False
2	103	charlie@ok.org	True
3	104	diana@@bad.com	False
4	105	eve@valid.net	True
5	106		False
6	107	frank@test	False

❌ Strategy 2: Record Rejection (Quarantine)

When a record is so corrupted that **correction would introduce false data**, the safest action is to remove it from the training set.

> ⚠️ **Rejection ≠ Deletion** — Rejected records go to a **quarantine file** for human review, not the trash!

When to reject:

- Multiple critical fields are *simultaneously* invalid
- Correct value cannot be inferred from context
- The record would **mislead any model** trained on it

```
🚫 Quarantine file preview:
   customer_id             name  age  salary          email                              rejection_reason
1          102        BOB JONES  -30   -5000   bob.email.com  invalid_age; invalid_salary; invalid_email
2          103  Charlie Brown    150   75000  charlie@ok.org                                  invalid_age
5          106       EVE MILLER   28  120000                                               invalid_email
6          107            frank  200   65000      frank@test                                  invalid_age
```

```python
df2 = customers_raw.copy()

# ── Define rejection mask (boolean conditions combined with | ) ──
reject_mask = (
    (df2['age'] < 0)  # impossible age
    | (df2['age'] > 120)  # impossible age
    | (df2['salary'] < 0)  # impossible salary
    | (~df2['email'].str.contains('@', na=False))  # invalid email
    | (df2['customer_id'].isnull())  # no identifier
)

# ── Split into clean vs. rejected ──────────────────────────────
df_clean = df2[~reject_mask].copy()
df_rejected = df2[reject_mask].copy()
```

```python
# NOTE:
# pd.isna() == pd.isnull()
# pd.notna() == pd.notnull()
```

```python
# ── Add human-readable rejection reason ────────────────────────
def build_reason(row):
    reasons = []
    if pd.notna(row['age']) and (row['age'] < 0 or row['age'] > 120):
        reasons.append('invalid_age')
    if pd.notna(row['salary']) and row['salary'] < 0:
        reasons.append('invalid_salary')
    if '@' not in str(row['email']):
        reasons.append('invalid_email')
    if pd.isna(row['customer_id']):
        reasons.append('missing_id')
    return '; '.join(reasons) if reasons else 'unknown'
```

```python
df_rejected['rejection_reason'] = df_rejected.apply(build_reason, axis=1)
```

Side Notes

Here axis refers to the **direction of travel** — axis=0 moves down (along rows), axis=1 moves across (along columns).

`.apply()` uses the same logic — axis means direction of travel, not what you receive:

```
df.apply(func, axis=0)  # func travels DOWN each column
                        # func receives one column at a time (a Series of row values)

df.apply(func, axis=1)  # func travels ACROSS each row
                        # func receives one row at a time (a Series of column values)
```

So `axis=1` means **"move across columns"** — which means you're processing **one row at a time**.

DataFrame `.apply()` — what our code uses:

```
df_rejected.apply(build_reason, axis=1)
```

- Called on the **whole DataFrame**
- `axis=1` means iterate **row by row**
- Each row is passed as a **Series** to the function, where the index is the column names
- So inside `build_reason`, `row['age']`, `row['salary']` etc. all work

```
row received by build_reason:
age            25
salary       5000
email     a@b.com
customer_id   101
Name: 0, dtype: object   ← a Series representing one row
```

Column `.apply()` — called on a single column:

```
df['age'].apply(some_function)
```

- Called on **one Series (column)**
- No `axis` parameter needed — it always iterates element by element
- Each individual **value** (not a row) is passed to the function

```
df['age'].apply(lambda x: x * 2)
# x receives: 25, then 30, then 40 ...  ← a scalar each time
```

2. 🔄 Handling Consistency & Schema Issues

Definition: The same real-world entity is represented in **multiple incompatible ways** across records, systems, or time periods.

Three enforcing strategies:

Strategy	What it does
Standardization	Convert all representations to one canonical form
Type Coercion	Enforce the correct data type per column
Schema Enforcement	Validate the full structure before analysis

2.1 📐 Standardization

Convert *all* representations of a value to one canonical form before any analysis.

Steps:

1. Audit distinct values (`value_counts()`)
2. Define the target canonical form
3. Write a mapping or transformation function
4. Apply and verify no values were lost

📊 Raw Employee Data (with consistency problems):

	emp_id	hire_date	gender	country	height	is_active
0	1	2024-01-15	male	USA	175cm	yes
1	2	15/02/2024	Female	United States	5ft 11in	True
2	3	March 5 2024	M	US	1.8m	1
3	4	2024.04.20	f	united states	168	no
4	5	May-10-2024	1	UAE	6ft 0in	false
5	6	2024/06/01	woman	United Arab Emirates	1.65m	YES
6	7	07-25-2024	Man	uk	180cm	0
7	8	2024-08-30	FEMALE	United Kingdom	5ft 6in	NO

```python
df_emp = employees_raw.copy()
```

```python
# ──────────────────────────────────────────────────────────────
# Standardize DATES → ISO 8601 (YYYY-MM-DD)
# pd.to_datetime() auto-detects many formats
# ──────────────────────────────────────────────────────────────
df_emp['hire_date'] = pd.to_datetime(
    df_emp['hire_date'], format='mixed', dayfirst=False
)
df_emp['hire_date'] = df_emp['hire_date'].dt.strftime('%Y-%m-%d')
```

```
📅 Dates standardized to ISO 8601:
          Before       After
0     2024-01-15  2024-01-15
1      15/02/2024  2024-02-15
2   March 5 2024  2024-03-05
3     2024.04.20  2024-04-20
4    May-10-2024  2024-05-10
5     2024/06/01  2024-06-01
6     07-25-2024  2024-07-25
7     2024-08-30  2024-08-30
```

```python
# In pd.to_datetime():
# converts a string to datetime

# 1- infer_datetime_format:
# was removed in pandas 3.0 — it was deprecated in pandas 2.0 and fully dropped in 3.0.
# Fix — just remove it, pandas 3.0 always infers the format automatically

# 2- format='mixed':
# use it if your data has mixed formats (some rows DD/MM/YYYY, others YYYY-MM-DD etc.)

# 3- dayfirst=False:
# when a date is ambiguous like 01/02/03, treat first number as month not day (US convention).
# Set True for DD/MM/YYYY data

# .dt is the datetime accessor (like .str for strings).
# .strftime() converts datetime objects back to strings in a format you specify.
```

```python
# ──────────────────────────────────────────────────────────────────────
# Standardize GENDER → canonical labels via mapping dictionary
# ──────────────────────────────────────────────────────────────────────
# To see the different ways of representation
print(employees_raw['gender'].str.strip().str.lower().unique().tolist())
print()
# Create a mapping dictonary of all the ways
gender_map = {
    'male': 'Male',
    'm': 'Male',
    '1': 'Male',
    'man': 'Male',
    'female': 'Female',
    'f': 'Female',
    '2': 'Female',
    'woman': 'Female',
}
df_emp['gender'] = df_emp['gender'].str.strip().str.lower().map(gender_map)
# .map() takes a dictionary and replaces each value in the Series with its corresponding dictionary value.
```

```
['male', 'female', 'm', 'f', '1', 'woman', 'man']

♉ Gender standardized:
   Before   After
0    male    Male
1  Female  Female
2       M    Male
3       f  Female
4       1    Male
5   woman  Female
6     Man    Male
7  FEMALE  Female
```

```python
# ──────────────────────────────────────────────────────────────
# Standardize HEIGHT → convert all to centimeters
# ──────────────────────────────────────────────────────────────
def to_cm(value):
    """Convert any height representation to centimeters."""
    v = str(value).strip()
    if 'ft' in v:  # e.g. '5ft 11in' or '6ft 0in'
        parts = v.replace('in', '').split('ft')
        feet = float(parts[0].strip())
        inches = float(parts[1].strip()) if parts[1].strip() else 0
        return round(feet * 30.48 + inches * 2.54, 1)
    elif 'm' in v and 'cm' not in v:  # e.g. '1.8m'
        return round(float(v.replace('m', '')) * 100, 1)
    else:  # e.g. '175cm' or bare number
        return round(float(v.replace('cm', '')), 1)


df_emp['height'] = df_emp['height'].apply(to_cm)
df_emp = df_emp.rename(columns={'height': 'height_cm'})
```

✏️ Height converted to cm:

	Before	After (cm)
0	175cm	175.0
1	5ft 11in	180.3
2	1.8m	180.0
3	168	168.0
4	6ft 0in	182.9
5	1.65m	165.0
6	180cm	180.0
7	5ft 6in	167.6

```
employees_raw['height'].dtype = str
df_emp['height_cm'].dtype     = float64
```

```python
# ------------------------------------------------------------------------
# Standardize BOOLEAN fields
# ------------------------------------------------------------------------
# To see the different ways of representation
print(employees_raw['is_active'].str.strip().str.lower().unique().tolist())
print()
# Create a mapping dictonary of all the ways
bool_map = {
    'yes': True,
    'no': False,
    '1': True,
    '0': False,
    'true': True,
    'false': False,
}
df_emp['is_active'] = df_emp['is_active'].str.strip().str.lower().map(bool_map)
```

```
['yes', 'true', '1', 'no', 'false', '0']

✅ Boolean standardized:
  Before  After
0    yes   True
1   True   True
2      1   True
3     no  False
4  false  False
5    YES   True
6      0  False
7     NO  False
```

Original non-standardized DataFrame:

	emp_id	hire_date	gender	country	height	is_active
0	1	2024-01-15	male	USA	175cm	yes
1	2	15/02/2024	Female	United States	5ft 11in	True
2	3	March 5 2024	M	US	1.8m	1
3	4	2024.04.20	f	united states	168	no
4	5	May-10-2024	1	UAE	6ft 0in	false
5	6	2024/06/01	woman	United Arab Emirates	1.65m	YES
6	7	07-25-2024	Man	uk	180cm	0
7	8	2024-08-30	FEMALE	United Kingdom	5ft 6in	NO

✅ Final standardized DataFrame:

	emp_id	hire_date	gender	country	height_cm	is_active
0	1	2024-01-15	Male	United States	175.0	True
1	2	2024-02-15	Female	United States	180.3	True
2	3	2024-03-05	Male	United States	180.0	True
3	4	2024-04-20	Female	United States	168.0	False
4	5	2024-05-10	Male	United Arab Emirates	182.9	False
5	6	2024-06-01	Female	United Arab Emirates	165.0	True
6	7	2024-07-25	Male	United Kingdom	180.0	False
7	8	2024-08-30	Female	United Kingdom	167.6	False

2.2 🔧 Type Coercion

> Data loaded from CSV or JSON often has **everything as strings**. Type coercion enforces the correct data type for each column.
>
> **Key pandas parameters:**
>
> - `pd.to_numeric(series, errors='coerce')` — converts strings to numbers; unparseable values become `NaN`
> - `pd.to_datetime(series, errors='coerce')` — converts strings to datetime
> - `.astype(dtype)` — explicit type casting
> - `errors='coerce'` vs `errors='raise'` vs `errors='ignore'`

Common Problem	Before	After	Method
Number stored as string	`"42"`, `"N/A"`	`42`, `NaN`	`pd.to_numeric(..., errors='coerce')`
Date stored as string	`"2024-01-15"`	`Timestamp('2024-01-15')`	`pd.to_datetime(...)`
Float stored as int	`42.0`	`42`	`.astype('Int64')`
Bool stored as string	`"True"`	`True`	custom mapping or `.astype(bool)`

```
Original types:
order_id      str
age           str
salary        str
product_id    str
created_at    str
dtype: object
```

```
Original data:
  order_id      age     salary product_id         created_at
0      001       25    50000.0         10  2024-01-15 09:00
1      002       30      75000         20         2024-02-20
2      003      N/A  not_avail        NaN           bad-date
3      004       40      90000         40         2024-04-30
4      005  unknown      65000         50         2024-05-15
```

python

```python
df_orders['age'] = ['25', '30', 'abc', '45']   # original

pd.to_numeric(...)
# [25.0, 30.0, NaN, 45.0]   ← strings→floats, 'abc'→NaN

.astype('Int64')
# [25, 30, <NA>, 45]        ← floats→integers, NaN→<NA>
```

Why `Int64` (capital I) not `int64`? Regular `int64` can't hold `NaN` — it's a numpy type. `Int64` is pandas' nullable integer type that supports `<NA>`.

```python
# ── Numeric coercion ──────────────────────────────────────────────
# errors='coerce' → turns unparseable values into NaN (safe fallback)
# Int64 (capital I) supports NaN; regular int64 does NOT
# s = pd.array([1, 2, None], dtype='Int64')  # ✅ stays Int64, shows <NA>
# s = pd.array([1, 2, None], dtype='int64')  # ❌ error
df_orders['age'] = pd.to_numeric(df_orders['age'], errors='coerce').astype('Int64')
df_orders['salary'] = pd.to_numeric(df_orders['salary'], errors='coerce').astype(
    'float64'
)
```

```
Original types:
order_id      str
age           str
salary        str
product_id    str
created_at    str
dtype: object
```

```python
# ── Integer with fallback for NaN ─────────────────────────────────
# Regular int can't hold NaN, so we fill with sentinel -1 first
df_orders['product_id'] = df_orders['product_id'].fillna(-1).astype(int)
```

```python
# ── Datetime coercion ─────────────────────────────────────────────
df_orders['created_at'] = pd.to_datetime(
    df_orders['created_at'], format='mixed', dayfirst=False, errors='coerce'
)
```

```
✅ Corrected types:
order_id                 str
age                    Int64
salary               float64
product_id             int64
created_at    datetime64[us]
dtype: object
```

Original data:

	order_id	age	salary	product_id	created_at
0	001	25	50000.0	10	2024-01-15 09:00
1	002	30	75000	20	2024-02-20
2	003	N/A	not_avail	NaN	bad-date
3	004	40	90000	40	2024-04-30
4	005	unknown	65000	50	2024-05-15

Cleaned data:

	order_id	age	salary	product_id	created_at
0	001	25	50000.0	10	2024-01-15 09:00:00
1	002	30	75000.0	20	2024-02-20 00:00:00
2	003	<NA>	NaN	-1	NaT
3	004	40	90000.0	40	2024-04-30 00:00:00
4	005	<NA>	65000.0	50	2024-05-15 00:00:00

2.3 📋 Schema Enforcement

A **schema** defines the expected structure of a dataset: column names, data types, allowed value ranges, and required fields.

Enforcing schema **before any analysis** catches problems at the source.

Three levels of schema enforcement:

Level	Tool	Best For
Basic	Python dict + Pandas	Quick validation scripts
Row-level	Pydantic	API ingestion, rich per-field errors
Production	Great Expectations	Data pipelines, automated HTML reports

Level 1: Python Dictionary Schema

```python
# ── Define schema as a Python dictionary ──────────────────────────
SCHEMA = {
    'customer_id': {'dtype': 'int', 'nullable': False},
    'age': {'dtype': 'int', 'nullable': False, 'min': 0, 'max': 120},
    'salary': {'dtype': 'float', 'nullable': True, 'min': 0},
    'email': {'dtype': 'str', 'nullable': False, 'pattern': r'^[^@]+@[^@]+\.[^@]+$'},
    'gender': {
        'dtype': 'str',
        'nullable': True,
        'allowed': ['Male', 'Female', 'Other'],
    },
}
```

```python
def validate_schema(df, schema):
    """Validate a DataFrame against a schema dictionary.
    Parameters
    ----------
    df     : pd.DataFrame  — the dataset to validate
    schema : dict          — schema definition
    Returns
    -------
    list of violation strings (empty = all good)
    """
    violations = []
    for col, rules in schema.items():
        if col not in df.columns:
            violations.append(f"❌ MISSING COLUMN: {col}")
            continue
        # Null check
        null_count = df[col].isnull().sum()
        if not rules.get('nullable', True) and null_count > 0:
            violations.append(f"❌ {col}: {null_count} nulls in non-nullable column")
        # Min/Max
        if 'min' in rules:
            bad = (df[col].dropna() < rules['min']).sum()
            if bad:
                violations.append(f"❌ {col}: {bad} values below min={rules['min']}")
        if 'max' in rules:
            bad = (df[col].dropna() > rules['max']).sum()
            if bad:
                violations.append(f"❌ {col}: {bad} values above max={rules['max']}")
        # Allowed set
        if 'allowed' in rules:
            bad = (~df[col].dropna().isin(rules['allowed'])).sum()
            if bad:
                violations.append(f"❌ {col}: {bad} values not in {rules['allowed']}")
        # Regex pattern
        if 'pattern' in rules:
            bad = (~df[col].dropna().str.match(rules['pattern'])).sum()
            if bad:
                violations.append(f"❌ {col}: {bad} values fail pattern check")
    return violations
```

```python
# ── Test with a dataset that has violations ──────────────────────────
df_validate = pd.DataFrame(
    {
        'customer_id': [1, 2, None, 4, 5],
        'age': [25, -5, 30, 200, 40],
        'salary': [50000, -1000, 75000, None, 90000],
        'email': ['a@b.com', 'bad-email', 'c@d.org', 'x@@y.net', 'e@f.com'],
        'gender': ['Male', 'Female', 'Alien', 'Male', 'Other'],
    }
)

violations = validate_schema(df_validate, SCHEMA)
if violations:
    print(f"Found {len(violations)} violation(s):\n")
    for v in violations:
        print("  ", v)
else:
    print("✅ No violations found!")
```

```
Found 6 violation(s):

   ❌ customer_id: 1 nulls in non-nullable column
   ❌ age: 1 values below min=0
   ❌ age: 1 values above max=120
   ❌ salary: 1 values below min=0
   ❌ email: 2 values fail pattern check
   ❌ gender: 1 values not in ['Male', 'Female', 'Other']
```

Level 2: Pydantic Row-Level Validation

> **Pydantic** validates data at the *row level* using Python type hints. Ideal when data enters through an API.
>
> - Validators run automatically during model initialization
> - Error messages are **precise and per-field**
> - `Field(ge=0, le=120)` means `0 ≤ value ≤ 120` (`ge` = greater-than-or-equal, `le` = less-than-or-equal)

```
# pip install pydantic
import pydantic
print(pydantic.__version__)
```

```
2.12.5
```

```python
from pydantic import BaseModel, Field, field_validator, ValidationError
from typing import Optional
```

```python
class CustomerRecord(BaseModel):
    model_config = {'validate_assignment': True}
    customer_id: int = Field(..., ge=0)
    age: int = Field(..., ge=0, le=120)
    salary: Optional[float] = Field(None, ge=0)
    email: str
    gender: Optional[str] = None

    @field_validator('email')
    @classmethod
    def email_must_be_valid(cls, v):
        if not re.match(r'^[^@]+@[^@]+\.[^@]+$', v):
            raise ValueError(f"Invalid email format: {v}")
        return v.lower()

    @field_validator('gender')
    @classmethod
    def gender_must_be_allowed(cls, v):
        if v is not None and v not in ['Male', 'Female', 'Other']:
            raise ValueError(f"Gender must be Male/Female/Other, got: {v}")
        return v
```

```python
# Validate row by row — including rows with NaN customer_id
clean_rows, rejected_rows = [], []
for _, row in df_validate.iterrows():  # removed dropna — handle NaN id manually
    try:
        record = CustomerRecord(**row.to_dict())
        clean_rows.append(record.model_dump())
    except ValidationError as e:
        # Uncomment the following line to see the content of e.errors() list of dicts:
        # print(f'e.errors(): \n{e.errors()}\n')

        # include invalid value in each error message
        all_errors = '; '.join(
            [
                f"{err['loc'][0]}: {err['msg']} (got: {err.get('input', 'N/A')})"
                for err in e.errors()
            ]
        )
        rejected_rows.append({'row_index': row.name, 'errors': all_errors})
```

```python
print("\nRejected details:")
for r in rejected_rows:
    print(f"  Row Index {r['row_index']}:")
    for err in r['errors'].split('; '):
        print(f"    • {err}")
```

Trying to validate this df:

	customer_id	age	salary	email	gender
0	1.0	25	50000.0	a@b.com	Male
1	2.0	-5	-1000.0	bad-email	Female
2	NaN	30	75000.0	c@d.org	Alien
3	4.0	200	NaN	x@@y.net	Male
4	5.0	40	90000.0	e@f.com	Other

✅ Valid rows: 2
❌ Invalid rows: 3

Rejected details:

Row Index 1:

- age: Input should be greater than or equal to 0 (got: -5)
- salary: Input should be greater than or equal to 0 (got: -1000.0)
- email: Value error, Invalid email format: bad-email (got: bad-email)

Row Index 2:

- customer_id: Input should be a finite number (got: nan)
- gender: Value error, Gender must be Male/Female/Other, got: Alien (got: Alien)

Row Index 3:

- age: Input should be less than or equal to 120 (got: 200)
- salary: Input should be greater than or equal to 0 (got: nan)
- email: Value error, Invalid email format: x@@y.net (got: x@@y.net)

Side Notes

`subset` tells `.dropna()` to **only look at specific columns** when deciding which rows to drop, instead of checking all columns.

```
df_validate.dropna(subset=['customer_id'])
# drop rows where customer_id is NaN
# ignore NaN in all other columns
```

Without `subset`:

```
df_validate.dropna()
# drops any row that has NaN in ANY column
```

Visually:

```
   customer_id   age   email
0          101    25     NaN   ← kept   (NaN in email, not customer_id)
1          NaN    30   a@b.c   ← dropped (NaN in customer_id)
2          103   NaN     NaN   ← kept   (NaN in age/email, not customer_id)
3          NaN   NaN     NaN   ← dropped (NaN in customer_id)
```

`subset` accepts a list so you can check multiple columns:

```
df_validate.dropna(subset=['customer_id', 'email'])
# drops rows where customer_id OR email is NaN
```

By default it's **OR** — drops a row if **any** of the subset columns is NaN. You can change this with `how`:

```
df_validate.dropna(subset=['customer_id', 'email'], how='any')
# drops row if customer_id NaN OR email NaN  ← default

df_validate.dropna(subset=['customer_id', 'email'], how='all')
# drops row only if customer_id NaN AND email NaN
```

Side Notes

```
record = CustomerRecord(**row.to_dict())
clean_rows.append(record.model_dump())
```

`row.to_dict()`

Breaking down every piece:

`row` is a pandas Series (one row from `.iterrows()`). `.to_dict()` converts it to a plain Python dictionary where keys are column names and values are the cell values:

```
row.to_dict()
# {
#   'customer_id': 101,
#   'age': 25,
#   'salary': 5000.0,
#   'email': 'alice@example.com',
#   'gender': 'Female'
# }
```

`**` — dictionary unpacking

`**` unpacks a dictionary into **keyword arguments**. So:

```
CustomerRecord(**row.to_dict())
```

is exactly equivalent to:

```
CustomerRecord(
    customer_id=101,
    age=25,
    salary=5000.0,
    email='alice@example.com',
    gender='Female'
)
```

`**` just saves you from writing each argument manually — it spreads the dictionary keys as argument names and values as argument values.

```
record = CustomerRecord(**row.to_dict())
clean_rows.append(record.model_dump())
```

Breaking down every piece:

`CustomerRecord(**row.to_dict())`

This creates a **Pydantic model instance** from the row data. At this moment Pydantic:

```
1. receives all field values
2. checks types      → is age an int? is email a str?
3. runs Field rules  → is age between 0 and 120?
4. runs validators   → does email match regex? is gender allowed?
5. if all pass       → creates the object ✅
6. if any fail       → raises ValidationError ❌
```

`record.model_dump()`

Converts the validated Pydantic model instance back to a plain Python dictionary:

```
record.model_dump()
# {
#   'customer_id': 101,
#   'age': 25,
#   'salary': 5000.0,
#   'email': 'alice@example.com',  ← normalized to lowercase by validator
#   'gender': 'Female'
# }
```

This is the **clean, validated version** of the row — types are correct, email is normalized, all rules passed.

Side Notes

```
try:
    record = CustomerRecord(**row.to_dict())
    clean_rows.append(record.model_dump())
except ValidationError as e:
```

`e.errors()` returns a **list of all errors** found in the row — Pydantic validates every field before raising, so all failures are already collected:

```
e.errors()
# [
#   {'loc': ('age',),   'msg': 'Input should be less than or equal to 120',
'input': 200, ... other items},
#   {'loc': ('email',), 'msg': 'Invalid email format: notanemail', 'input':
'notanemail', ... other items},
# ]
```

Then:

```
[f"{err['loc'][0]}: {err['msg']} (got: {err.get('input', 'N/A')})" for err in
e.errors()]
# ['age: Input should be less than or equal to 120 (got: 200)',
#  'email: Invalid email format: notanemail (got: notanemail)']
```

3. Handling Data Completeness Issues (Missing Data)

> Missing data is not just `NaN`! It appears in many disguises.

3.1 Forms of Missing Data

Form	Examples	How to Detect
Explicit	`NaN`, `None`, `NULL`, `NA`	`df.isnull()`
Implicit placeholders	`""`, `" "`, `"?"`, `"N/A"`, `"Unknown"`, `"-"`	`str.strip()` + custom check
Sentinel values	`-999`, `-1`, `9999`, `0`	Domain knowledge
Structural	Pregnancy data for male patients	Domain knowledge
Temporal	Future events not yet recorded	Context

```python
# Dataset demonstrating ALL forms of missing data
df_missing_forms = pd.DataFrame(
    {
        'id': [1, 2, 3, 4, 5, 6, 7, 8],
        'score': [85, None, -999, 92, 0, 78, np.nan, 88],
        'comment': ['Good', '', ' ', '?', 'N/A', 'Excellent', 'Unknown', '-'],
        'income': [50000, 75000, None, 9999, np.nan, 60000, 80000, None],
        'birth_year': [1990, 1985, None, 9999, 1978, None, 1992, 1988],
    }
)
```

```python
# Detect explicit NaN
print("\n🔹 Explicit NaN count per column:")
print(df_missing_forms.isnull().sum())
```

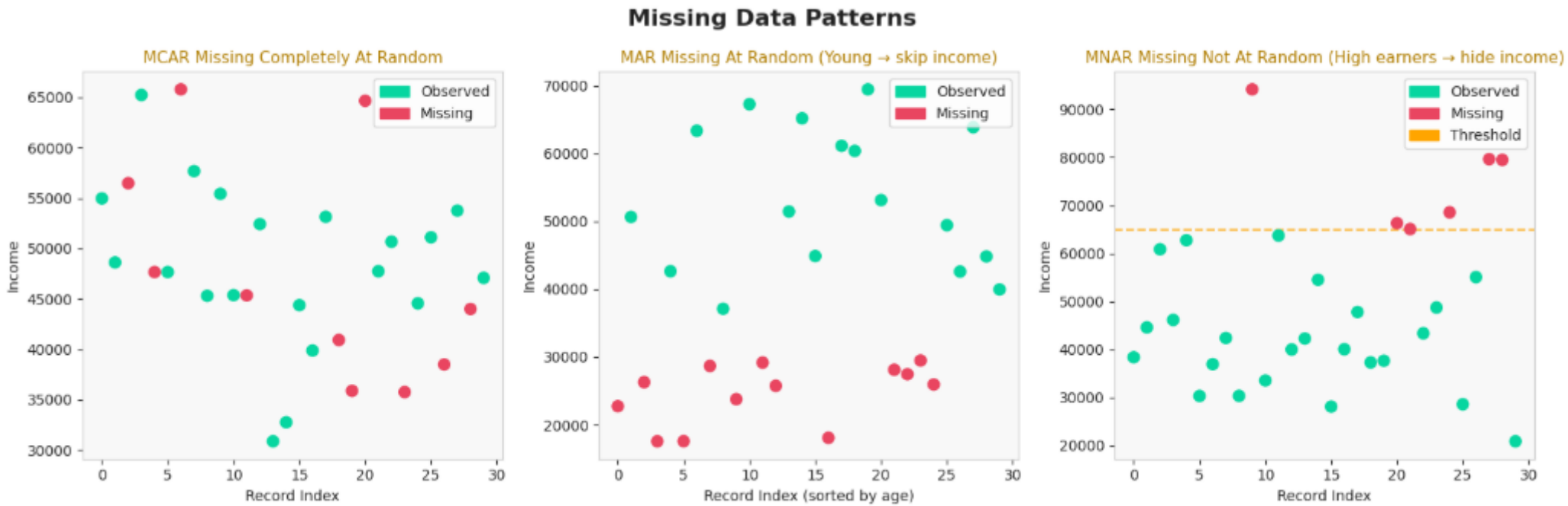
```python
# Detect implicit placeholders in string column
placeholder_values = ['', ' ', '?', 'N/A', 'Unknown', '-', 'null', 'none']
df_missing_forms['comment_placeholder'] = (
    df_missing_forms['comment'].str.strip().isin(placeholder_values)
)
```

```python
# Detect sentinel values
SENTINEL_SCORE = -999
SENTINEL_YEAR = 9999
df_missing_forms['score_sentinel'] = df_missing_forms['score'] == SENTINEL_SCORE
df_missing_forms['year_sentinel'] = df_missing_forms['birth_year'] == SENTINEL_YEAR
```

3.2 Missing Data Patterns

> ⚠️ **Understanding the pattern matters!** The right treatment depends on *why* data is missing.

Pattern	Full Name	Definition	Example	Danger
MCAR	Missing Completely At Random	No relationship between missingness and data	Survey response lost due to server crash	Low — safe to delete
MAR	Missing At Random	Missingness depends on *other observed* variables	Younger people skip income field	Medium — impute carefully
MNAR	Missing Not At Random	Missingness depends on the *missing value itself*	High earners skip income field	High — no easy fix



Type	Why missing	Depends on	Difficulty
MCAR	pure chance	nothing	easy
MAR	another observed column	age, gender etc.	manageable
MNAR	the missing value itself	the hidden income	hard

MCAR — Missing Completely At Random (left plot)

```
Missing because: pure random chance
```

MAR — Missing At Random (middle plot)

```
Missing because: depends on another observed column
```

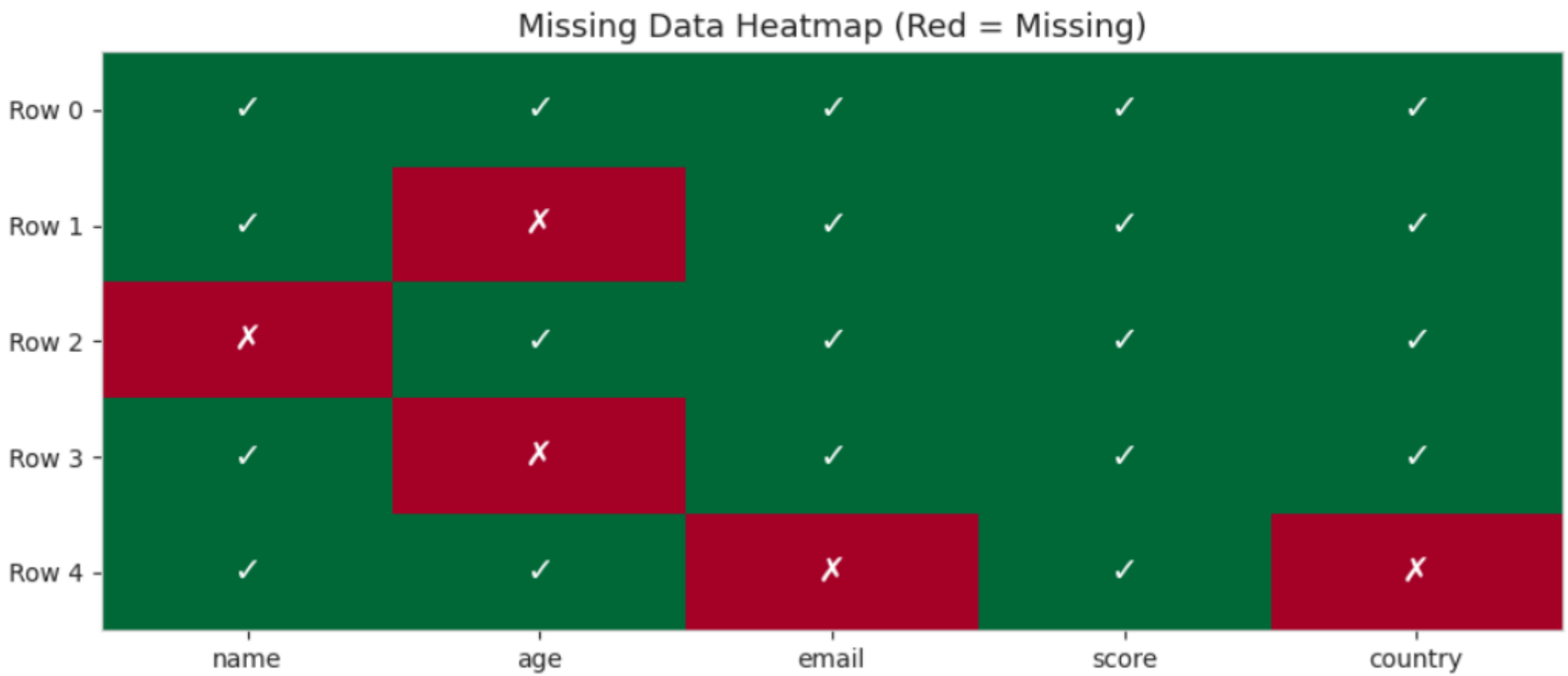
MNAR — Missing Not At Random (right plot)

```
Missing because: depends on the hidden value itself
```

3.3 Missing Data Detection

1️⃣ Explicit NaN (df.isnull().sum()):

```
name       1
age        2
email      1
score      0
country    1
```



3.4 Handling Missing Data — Deletion

> **When to delete:**
>
> - `<5%` missing values (**rule of thumb**)
> - Pattern is **MCAR** (safe to drop without bias)
>
> ⚠️ **Risk:** May introduce bias if data is NOT MCAR.

```
# ── dropna() reference ──────────────────────────────────────────
# Signature: df.dropna(axis=0, how='any', thresh=None, subset=None)
#
# axis   : 0 = drop rows (default), 1 = drop columns
# how    : 'any' = drop if ANY NaN, 'all' = only drop if ALL NaN
# thresh : keep rows with AT LEAST this many "non"-NaN values
# subset : list of columns to check for NaN (only those matter)
```

```python
print("1-  dropna()  → remove ALL rows with ANY missing value:")
print(df_del.dropna())
print()

print("2-  dropna(how='all') → remove rows where ALL values are NaN:")
print(df_del.dropna(how='all'))
print()

print("3-  dropna(subset=['A','B']) → remove rows missing in A OR B:")
print(df_del.dropna(subset=['A', 'B']))
print()

print("4-  dropna(subset=['A','B'], how='all') → remove rows missing in A AND B:")
print(df_del.dropna(subset=['A', 'B'], how='all'))
print()

print("5-  dropna(thresh=3) → keep rows with at least 3 non-null values:")
print(df_del.dropna(thresh=3))
print()

print("6-  Drop COLUMNS with >50% missing:")
threshold = 0.5
df_col_drop = df_del.loc[:, df_del.isnull().mean() < threshold]
print(
    f"   Dropped columns: {list(df_del.columns[df_del.isnull().mean() >= threshold])}"
)
print(df_col_drop)
```

The dropped columns print:

```python
df_del.columns[df_del.isnull().mean() >= threshold]
# flipped condition (>=) → shows exactly what was dropped
```

3.5 Simple Imputation

`sklearn.impute.SimpleImputer`

```
from sklearn.impute import KNNImputer, SimpleImputer
```

Parameter	Values	Effect
`strategy`	`'mean'`	Replace with column mean — use for normal distributions
`strategy`	`'median'`	Replace with median — robust to outliers
`strategy`	`'most_frequent'`	Use mode — best for categorical
`strategy`	`'constant'`	Use `fill_value` — when 0 or "Unknown" makes sense

```
# ── Mean Imputation (sensitive to outliers) ──────────────────────
imp_mean = SimpleImputer(strategy='mean')
age_mean = imp_mean.fit_transform(df_imp[['age']])
salary_mean = SimpleImputer(strategy='mean').fit_transform(df_imp[['salary']])

# ── Median Imputation (robust to outliers) ──────────────────────
imp_median = SimpleImputer(strategy='median')
age_median = imp_median.fit_transform(df_imp[['age']])
salary_median = SimpleImputer(strategy='median').fit_transform(df_imp[['salary']])

# ── Mode Imputation (for categorical) ──────────────────────
imp_mode = SimpleImputer(strategy='most_frequent')
dept_mode = imp_mode.fit_transform(df_imp[['dept']])
```

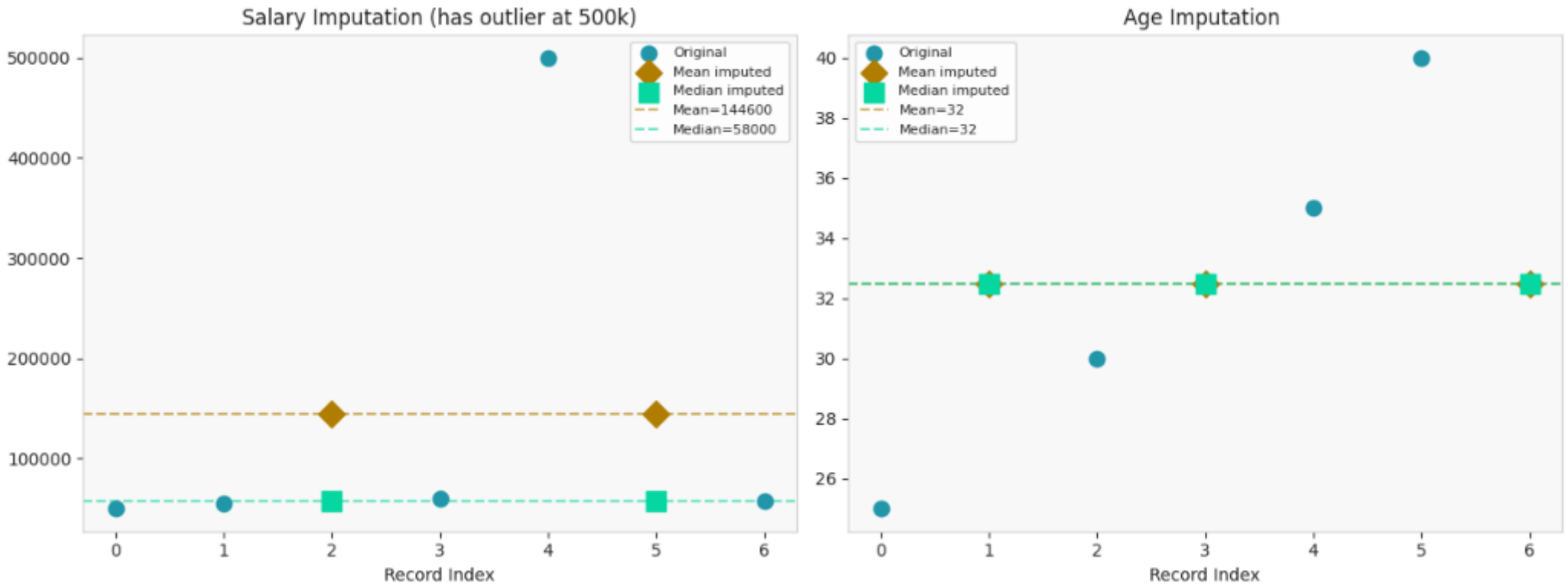
```python
print("Imputation comparison:")
comparison = pd.DataFrame(
    {
        'name': df_imp['name'],
        'age_original': df_imp['age'],
        'age_mean': age_mean.flatten().round(1),
        'age_median': age_median.flatten().round(1),
        'salary_orig': df_imp['salary'],
        'salary_mean': salary_mean.flatten().round(0),
        'salary_median': salary_median.flatten().round(0),
        'dept_original': df_imp['dept'],
        'dept_mode': dept_mode.flatten(),
    }
)
```

Why .flatten()?

> `fit_transform()` returns a **2D numpy array**, even when you give it a single column.
>
> But `pd.DataFrame({...})` expects a **1D array** for each column.
>
> `.flatten()` simply collapses the 2D array into 1D so it fits cleanly as a DataFrame column.

```
Imputation comparison:
      name  age_original  age_mean  age_median  salary_orig  salary_mean  salary_median dept_original dept_mode
0    Alice          25.0      25.0        25.0      50000.0      50000.0        50000.0         Sales     Sales
1      Bob           NaN      32.5        32.5      55000.0      55000.0        55000.0           NaN        HR
2  Charlie          30.0      30.0        30.0          NaN     144600.0        58000.0         Sales     Sales
3    Diana           NaN      32.5        32.5      60000.0      60000.0        60000.0            HR        HR
4      Eve          35.0      35.0        35.0     500000.0     500000.0       500000.0           NaN        HR
5    Frank          40.0      40.0        40.0          NaN     144600.0        58000.0            HR        HR
6    Grace           NaN      32.5        32.5      58000.0      58000.0        58000.0            HR        HR
```

⚠️ Notice: salary_mean fills missing with $144,500 (pulled by outlier)
salary_median fills with $57,500 (unaffected by outlier) — much better!


Salary Imputation (has outlier at 500k)
Original
Mean imputed
Median imputed
Mean=144600
Median=58000
500000
400000
300000
200000
100000
0
1
2
3
4
5
6
Record Index
Age Imputation
Original
Mean imputed
Median imputed
Mean=32
Median=32
40
38
36
34
32
30
28
26
0
1
2
3
4
5
6
Record Index

4. 🔁 Handling Data Uniqueness Issues (Duplicates)

Duplicate records are rows containing identical or highly similar information.

Sources: Multiple data entry, system errors, data merging, user resubmissions.

Types:

Type	Definition	Example
Exact duplicates	All field values identical	Two identical rows
Subset duplicates	Key fields match, others differ	Same `customer_id`, different timestamps
Fuzzy duplicates	Tiny variations	`"jon smith"` vs `"john smith"`

```python
# ── 1. Removing exact duplicates ──────────────────────────────────────
# drop_duplicates(subset, keep)
# subset : columns to check (None = all columns)
# keep   : 'first', 'last', False (drop ALL occurrences)

print("\n1️⃣  Keep FIRST occurrence of exact duplicates:")
print("df_dupes.drop_duplicates(keep='first')")
print(df_dupes.drop_duplicates(keep='first'))

print("\n2️⃣  Keep LAST occurrence:")
print("df_dupes.drop_duplicates(keep='last')")
print(df_dupes.drop_duplicates(keep='last'))

print("\n3️⃣  Remove based on customer_id — keep first transaction:")
print("df_dupes.drop_duplicates(subset=['customer_id'], keep='first')")
print(df_dupes.drop_duplicates(subset=['customer_id'], keep='first'))

print("\n4️⃣  Remove ALL duplicates (keep=False) — only unique rows survive:")
print("df_dupes.drop_duplicates(keep=False)")
print(df_dupes.drop_duplicates(keep=False))
```

ORIGINAL:

	customer_id	name	email	purchase_amount	date
0	1	Alice	a@b.com	100	2024-01-01
1	2	Bob	b@c.com	250	2024-01-02
2	2	Bob	b@c.com	250	2024-01-02
3	3	Charlie	c@d.com	300	2024-01-03
4	4	Diana	d@e.com	50	2024-01-04
5	4	Diana	d@e.com	75	2024-01-05
6	4	Diana	d@e.com	120	2024-01-06
7	5	Eve	e@f.com	400	2024-01-07

2️⃣ Keep LAST occurrence:

```
df_dupes.drop_duplicates(keep='last')
```

	customer_id	name	email	purchase_amount	date
0	1	Alice	a@b.com	100	2024-01-01
2	2	Bob	b@c.com	250	2024-01-02
3	3	Charlie	c@d.com	300	2024-01-03
4	4	Diana	d@e.com	50	2024-01-04
5	4	Diana	d@e.com	75	2024-01-05
6	4	Diana	d@e.com	120	2024-01-06
7	5	Eve	e@f.com	400	2024-01-07

3️⃣ Remove based on customer_id — keep first transaction:

```
df_dupes.drop_duplicates(subset=['customer_id'], keep='first')
```

	customer_id	name	email	purchase_amount	date
0	1	Alice	a@b.com	100	2024-01-01
1	2	Bob	b@c.com	250	2024-01-02
3	3	Charlie	c@d.com	300	2024-01-03
4	4	Diana	d@e.com	50	2024-01-04
7	5	Eve	e@f.com	400	2024-01-07

4️⃣ Remove ALL duplicates (keep=False) — only unique rows survive:

```
df_dupes.drop_duplicates(keep=False)
```

	customer_id	name	email	purchase_amount	date
0	1	Alice	a@b.com	100	2024-01-01
3	3	Charlie	c@d.com	300	2024-01-03
4	4	Diana	d@e.com	50	2024-01-04
5	4	Diana	d@e.com	75	2024-01-05
6	4	Diana	d@e.com	120	2024-01-06
7	5	Eve	e@f.com	400	2024-01-07

```python
# — 2. Aggregating duplicates (preserve information) ————————————————
# Better when duplicates represent multiple transactions per customer

print("\n  Aggregate duplicates — combine multiple transactions per customer:")
aggregated = (
    df_dupes.groupby(['customer_id', 'name', 'email'])
    .agg(
        total_purchase=('purchase_amount', 'sum'),
        num_transactions=('purchase_amount', 'count'),
        avg_purchase=('purchase_amount', 'mean'),
        first_date=('date', 'min'),
        last_date=('date', 'max'),
    )
    .reset_index()
)
```

`.groupby(['customer_id', 'name', 'email'])`

Groups rows that share the same `customer_id`, `name`, and `email` together — all of Diana's 3 rows become one group, all of Bob's 2 rows become one group, etc.

`.agg(...)`

Applies aggregation functions to each group. Each argument follows the pattern:

```
new_column_name = ('existing_column', 'aggregation_function')
```

```
total_purchase  = ('purchase_amount', 'sum')    # sum all purchases per customer
```

`.reset_index()`

After `groupby + agg`, the group keys (`customer_id`, `name`, `email`) become the index. `.reset_index()` promotes them back to regular columns and resets the index to 0, 1, 2....

ORIGINAL:

	customer_id	name	email	purchase_amount	date
0	1	Alice	a@b.com	100	2024-01-01
1	2	Bob	b@c.com	250	2024-01-02
2	2	Bob	b@c.com	250	2024-01-02
3	3	Charlie	c@d.com	300	2024-01-03
4	4	Diana	d@e.com	50	2024-01-04
5	4	Diana	d@e.com	75	2024-01-05
6	4	Diana	d@e.com	120	2024-01-06
7	5	Eve	e@f.com	400	2024-01-07

```python
aggregated = (
    df_dupes.groupby(['customer_id', 'name', 'email'])
    .agg(
        total_purchase=('purchase_amount', 'sum'),
        num_transactions=('purchase_amount', 'count'),
        avg_purchase=('purchase_amount', 'mean'),
        first_date=('date', 'min'),
        last_date=('date', 'max'),
    )
    .reset_index()
)
```

Aggregated (one row per customer with summary stats):

	customer_id	name	email	total_purchase	num_transactions	avg_purchase	first_date	last_date
0	1	Alice	a@b.com	100	1	100.000000	2024-01-01	2024-01-01
1	2	Bob	b@c.com	500	2	250.000000	2024-01-02	2024-01-02
2	3	Charlie	c@d.com	300	1	300.000000	2024-01-03	2024-01-03
3	4	Diana	d@e.com	245	3	81.666667	2024-01-04	2024-01-06
4	5	Eve	e@f.com	400	1	400.000000	2024-01-07	2024-01-07

→ Diana made 3 transactions: $50, $75, $120 → total $245

5. 📈 Handling Data Outlier Issues

An **outlier** is a data point that differs significantly from other observations.

Outliers can be:

- 🐛 **Errors** → data entry mistakes, sensor malfunctions (should be removed/corrected)
- 🌟 **Genuine extreme values** → a billionaire in an income survey (should be kept or capped)

5.1 Detection Methods

Method 1: IQR (Interquartile Range)

Detects outliers using **quartile spread** — robust to outliers themselves!

- **Q1** = 25th percentile, **Q3** = 75th percentile
- **IQR** = Q3 − Q1
- Outlier if: `value < Q1 - 1.5×IQR` or `value > Q3 + 1.5×IQR`
- Use `3×IQR` for extreme outliers only

```
# ── IQR Method ──────────────────
Q1 = prices.quantile(0.25)
Q3 = prices.quantile(0.75)
IQR = Q3 - Q1
lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR
```

```
📐 IQR Method:
  Q1=11.0, Q3=13.0, IQR=2.0
  Lower fence (Q1 - 1.5×IQR): 8.0
  Upper fence (Q3 + 1.5×IQR): 16.0

Outliers detected by IQR: [200 -50]
```

```python
# ── Z-Score Method ──────────────────────────────────────────────
from scipy import stats

z_scores = np.abs(stats.zscore(prices))
threshold = 3  # standard choice: z > 3 is an outlier
print(f'z_scores[z_scores > threshold] = {z_scores[z_scores > threshold]}')
outliers_z = prices[z_scores > threshold]
```

```
z_scores[z_scores > threshold]= [4.14508655]
📊 Z-Score Method (threshold = 3):
  Z-scores: [0.19 0.14 0.17 0.12 0.19 0.1  0.14 0.17 0.12 0.08 0.14 4.15 0.17 0.12
 0.14 0.19 0.1  1.56 0.14 0.17]
  Outliers detected: [200]

💡 Z-score = (value - mean) / std
   Value 200: z = (200 - 18.3) / 45.0 = 4.0
```

5.2 Outlier Treatment Strategies

Strategy	When to Use	Effect
Removal	Confirmed errors, impossible values	Smaller dataset, no distortion
Capping (Winsorizing)	Real extremes we want to limit	Preserves sample size
Transformation	Positive skew, right-tailed distributions	Compresses scale

```
# ── Strategy 1: Removal ──────────────────────────────────────────
print('\n── Strategy 1: Removal ──────────────────────────────────────────')
Q1 = df_price['price'].quantile(0.25)
Q3 = df_price['price'].quantile(0.75)
IQR = Q3 - Q1
df_price['is_outlier'] = (df_price['price'] < Q1 - 1.5 * IQR) | (
    df_price['price'] > Q3 + 1.5 * IQR
)
df_removed = df_price[~df_price['is_outlier']].copy()
```

```
── Strategy 1: Removal ──────────────────────────

1  After REMOVAL: 14 → 12 rows
   Removed:
   product  price
6        G    200
12       M    -50
```

```python
# ── Strategy 2: Capping (Winsorization) ──────────────────────────────
print('\n── Strategy 2: Capping (Winsorization) ──────────────────────────────')
# clip(lower, upper) → values below lower become lower; above upper become upper
lower_cap = df_price['price'].quantile(0.05)
upper_cap = df_price['price'].quantile(0.95)
df_price['price_capped'] = df_price['price'].clip(lower=lower_cap, upper=upper_cap)
```

```
2️⃣  CAPPING (5th-95th percentile: [-12.3, 79.1]):
   product  price  price_capped
0        A     10          10.0
1        B     12          12.0
2        C     11          11.0
3        D     13          13.0
4        E      9           9.0
5        F     14          14.0
6        G    200          79.1
7        H     11          11.0
8        I     13          13.0
9        J     10          10.0
10       K     12          12.0
11       L      8           8.0
12       M    -50         -12.3
13       N     13          13.0
```

```python
# ── Strategy 3: Transformation ──────────────────────────────────────
print('\n── Strategy 3: Transformation ──────────────────────────────────────')
from scipy.stats import boxcox

# Work with positive values only for transformations
positive_prices = df_price['price'].clip(lower=0.01)  # avoid log(0)
df_price['price_log'] = np.log1p(positive_prices)  # log(1+x) safe for 0
df_price['price_sqrt'] = np.sqrt(positive_prices)
df_price['price_boxcox'], _ = boxcox(positive_prices + 1)
```

```
── Strategy 3: Transformation ──────────────────────────────────────

3️⃣  TRANSFORMATIONS (applied to positive values):
   product  price  price_log  price_sqrt  price_boxcox
0        A     10       2.40        3.16          2.26
1        B     12       2.56        3.46          2.40
2        C     11       2.48        3.32          2.33
3        D     13       2.64        3.61          2.47
4        E      9       2.30        3.00          2.17
5        F     14       2.71        3.74          2.53
6        G    200       5.30       14.14          4.64
7        H     11       2.48        3.32          2.33
8        I     13       2.64        3.61          2.47
9        J     10       2.40        3.16          2.26
10       K     12       2.56        3.46          2.40
11       L      8       2.20        2.83          2.08
12       M    -50       0.01        0.10          0.01
13       N     13       2.64        3.61          2.47
```

6. ⏱️ Handling Timeliness Issues — Filling Gaps in Time Series

Time series data often has **gaps** — missing readings at certain timestamps.

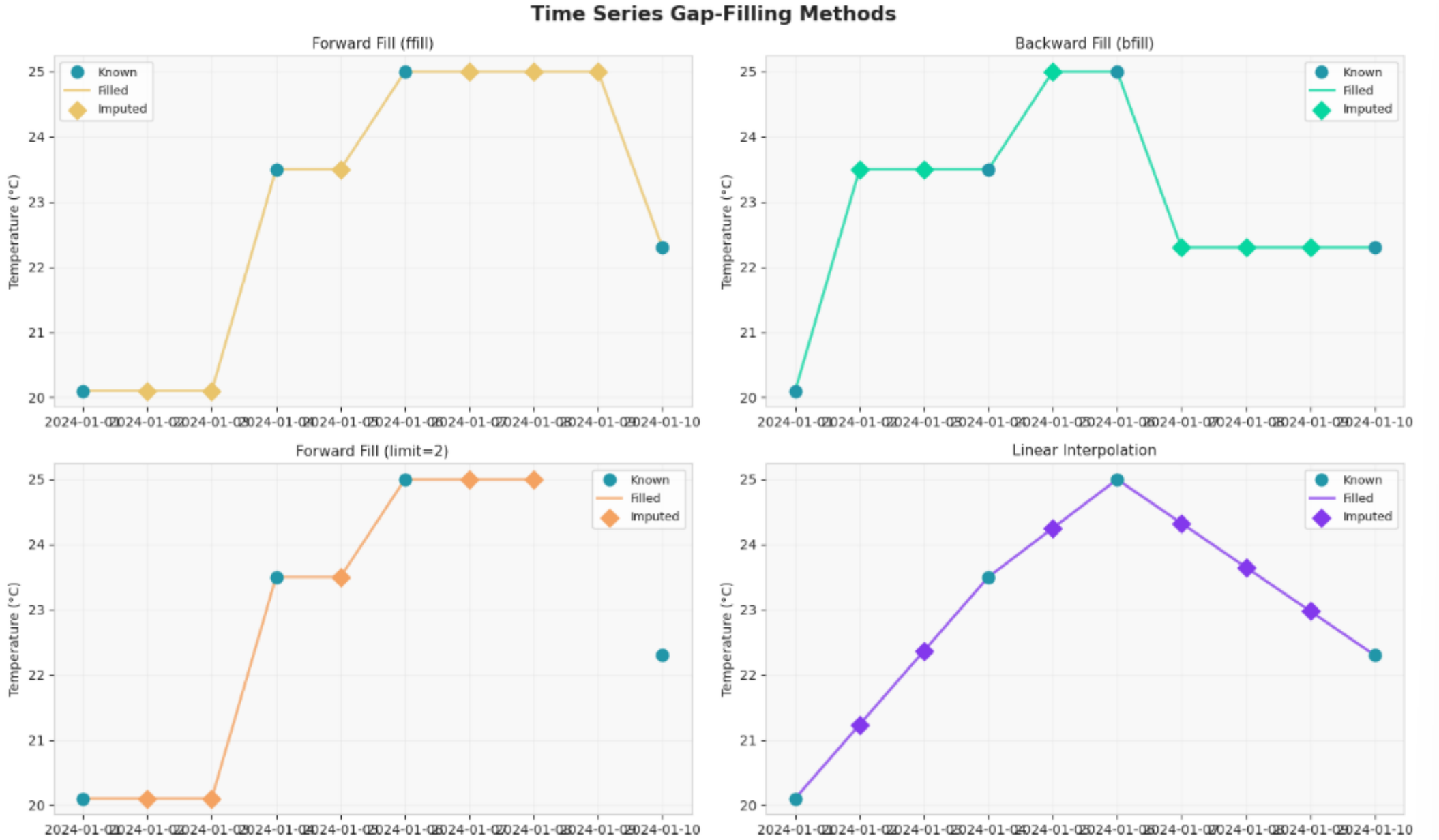
Four key filling methods:

Method	How it works	Best for
ffill (forward fill)	Copy last known value forward	Sensor readings, prices
bfill (backward fill)	Copy next known value backward	When future is known
limit	Restrict consecutive fills	Avoid filling too far
interpolate	Fit a line between known points	Smooth, continuous data

```python
# ── Method 1: Forward Fill ──────────────────────────────────────
# fillna(method='ffill') carries last known value forward
# ffill = temps.fillna(method='ffill')  # OLDER PANDAS
ffill = temps.ffill()

# ── Method 2: Backward Fill ─────────────────────────────────────
# bfill = temps.fillna(method='bfill')  # OLDER PANDAS
bfill = temps.bfill()

# ── Method 3: Limited Forward Fill ──────────────────────────────
# limit=2: fill at most 2 consecutive NaNs (avoids extrapolating too far)
# ffill_limited = temps.fillna(method='ffill', limit=2)  # OLDER PANDAS
ffill_limited = temps.ffill(limit=2)

# ── Method 4: Linear Interpolation ──────────────────────────────
# interpolate(method='linear') draws a straight line between known points
interp = temps.interpolate(method='linear')
# interpolate(method='linear') estimates missing values
# by drawing a straight line between the two nearest known value
# and filling in evenly spaced steps.
```

```python
comparison = pd.DataFrame(
    {
        'Original': temps,
        'ffill': ffill,
        'bfill': bfill,
        'ffill (limit=2)': ffill_limited,
        'interpolated': interp,
    }
)
```

```
Comparison of filling methods:
            Original  ffill  bfill  ffill (limit=2)  interpolated
2024-01-01      20.1   20.1   20.1             20.1     20.100000
2024-01-02       NaN   20.1   23.5             20.1     21.233333
2024-01-03       NaN   20.1   23.5             20.1     22.366667
2024-01-04      23.5   23.5   23.5             23.5     23.500000
2024-01-05       NaN   23.5   25.0             23.5     24.250000
2024-01-06      25.0   25.0   25.0             25.0     25.000000
2024-01-07       NaN   25.0   22.3             25.0     24.325000
2024-01-08       NaN   25.0   22.3             25.0     23.650000
2024-01-09       NaN   25.0   22.3              NaN     22.975000
2024-01-10      22.3   22.3   22.3             22.3     22.300000
```


Time Series Gap-Filling Methods
Forward Fill (ffill)
Backward Fill (bfill)
Forward Fill (limit=2)
Linear Interpolation
Known
Filled
Imputed
Temperature (°C)
20
21
22
23
24
25
2024-01-10

7. ⚖️ Handling Data Imbalance Issues

Class imbalance occurs when classes have very different frequencies.

Example: Fraud detection — 0.1% fraud, 99.9% legitimate transactions.

Why it matters:

- 🎯 **Model bias**: Classifiers favor the majority class
- 📉 **Poor minority recall**: Few examples to learn from
- 🚨 **Misleading accuracy**: 99.9% accuracy by predicting "not fraud" every time!

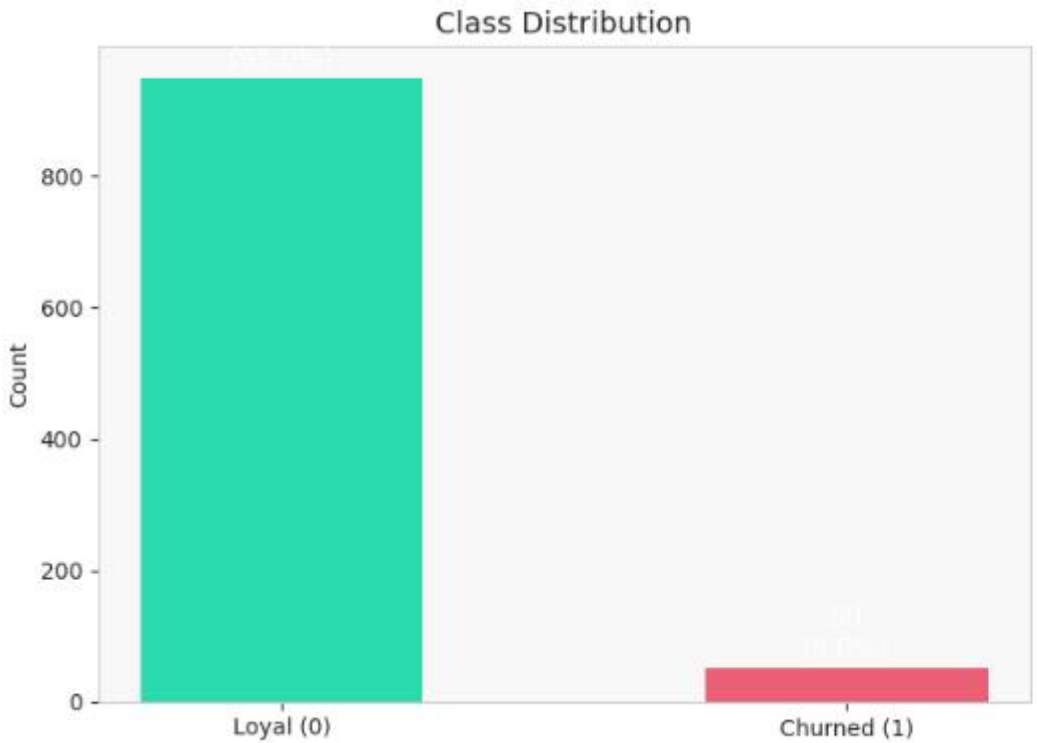
```
label
Loyal (0)      950
Churned (1)     50
Name: count, dtype: int64

Imbalance Ratio (IR) = 19.0:1  → Severe ⚠️
```

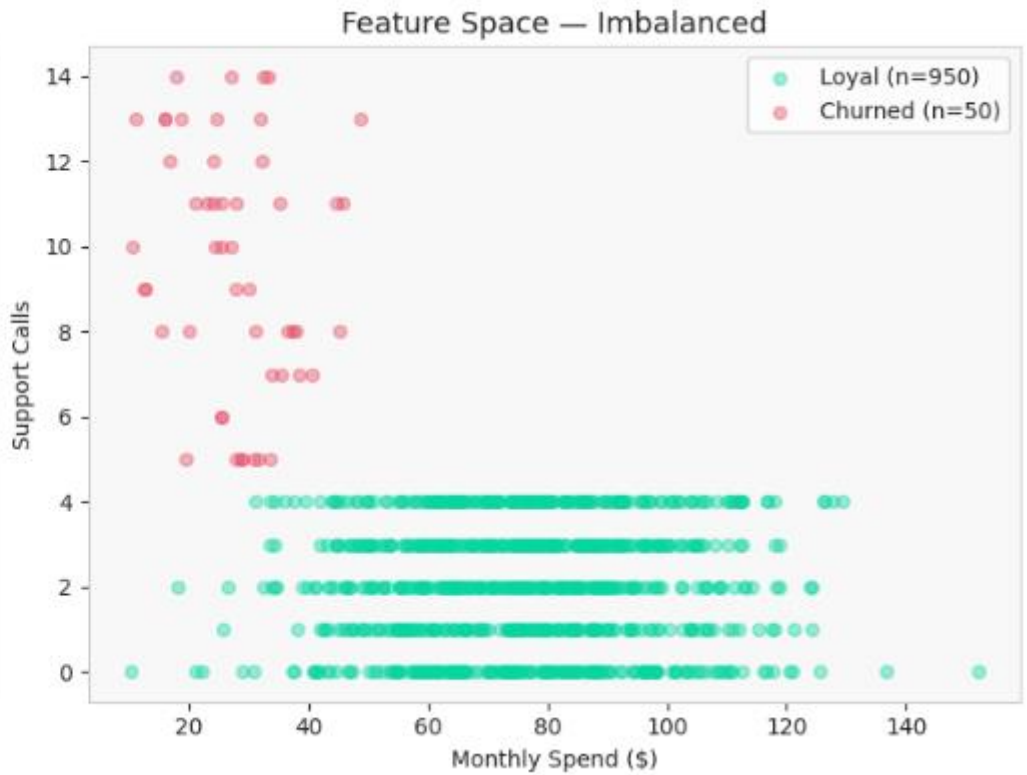
Imbalance Ratio (IR)

IR = (Majority class count) / (Minority class count)

IR	Level	Action
≤ 3:1	Mild	Often OK with standard methods
3:1 – 9:1	Moderate	Needs attention
≥ 9:1	Severe	Requires specialized techniques
> 100:1	Extreme	Treat as anomaly detection

```
label
Loyal (0)      950
Churned (1)     50
Name: count, dtype: int64

Imbalance Ratio (IR) = 19.0:1  → Severe ⚠️
```





7.1 Resampling Techniques

> ⚠️ **Critical Rule:** Apply resampling to **TRAINING data only!** Never to test data.

Techniques:

1. **Random Oversampling** — duplicate minority samples (fast, risk of overfitting)
2. **SMOTE** — create *synthetic* minority samples (best practice)
3. **Random Undersampling** — remove majority samples (loses information)
4. **Tomek Links** — remove majority samples near boundary (cleanup step)

```
# Uncomment if not installed:
# pip install imblearn
```

```
from imblearn.over_sampling import RandomOverSampler, SMOTE
from imblearn.under_sampling import RandomUnderSampler, TomekLinks
```

SMOTE — Synthetic Minority Oversampling Technique

A way to fix **class imbalance** by **creating synthetic new samples** of the minority class, rather than just duplicating existing ones.

How it works: Instead of copying existing minority rows, SMOTE **interpolates between them**:

```
Existing churner A:  [spend=20, tenure=5,  calls=8]
Existing churner B:  [spend=30, tenure=9,  calls=6]

Synthetic sample:    [spend=25, tenure=7,  calls=7]  ← point between A and B
```

It picks a real minority sample, finds its nearest neighbours, then creates a new point **somewhere along the line between them**.

Usage:

```python
from imblearn.over_sampling import SMOTE

sm = SMOTE(random_state=42)
X_resampled, y_resampled = sm.fit_resample(X, y)

# Before: {0: 950, 1: 50}
# After:  {0: 950, 1: 950}  ← minority class boosted to match majority
```

```python
unique, counts = np.unique(y, return_counts=True)
print(f"Before resampling: {dict(zip(unique.tolist(), counts.tolist()))}")
```

```python
# ── 1. Random Oversampling ──────────────────────────────────────
ros = RandomOverSampler(random_state=42)
X_ros, y_ros = ros.fit_resample(X, y)
print(f"\n1  Random Oversampling: {dict(zip(*np.unique(y_ros, return_counts=True)))}")

# ── 2. SMOTE ────────────────────────────────────────────────────
smote = SMOTE(k_neighbors=5, random_state=42)
X_smote, y_smote = smote.fit_resample(X, y)
print(f"2  SMOTE:              {dict(zip(*np.unique(y_smote, return_counts=True)))}")

# ── 3. Random Undersampling ─────────────────────────────────────
rus = RandomUnderSampler(random_state=42)
X_rus, y_rus = rus.fit_resample(X, y)
print(f"3  Random Undersampling: {dict(zip(*np.unique(y_rus, return_counts=True)))}")
```

```
1  Random Oversampling: {np.int64(0): np.int64(950), np.int64(1): np.int64(950)}
2  SMOTE:               {np.int64(0): np.int64(950), np.int64(1): np.int64(950)}
3  Random Undersampling: {np.int64(0): np.int64(50), np.int64(1): np.int64(50)}
```

`print(f"Before resampling: {dict(zip(*np.unique(y, return_counts=True)))}")`

Breaking down the complex inner part:

`np.unique(y, return_counts=True)` Returns two arrays — unique values and their counts:

```
np.unique([0,0,0,1,1], return_counts=True)
# (array([0, 1]),       ← unique values
#  array([3, 2]))       ← their counts
```

`*` **(unpacking)** The `*` splits that tuple into two separate arrays so `zip` can receive them:

```
# Without *
zip( (array([0,1]), array([700,300])) )  # ❌ zip gets 1 argument (a tuple)

# With *
zip( array([0,1]), array([700,300]) )    # ✅ zip gets 2 separate arrays
```

`zip(...)` Pairs up values with their counts:

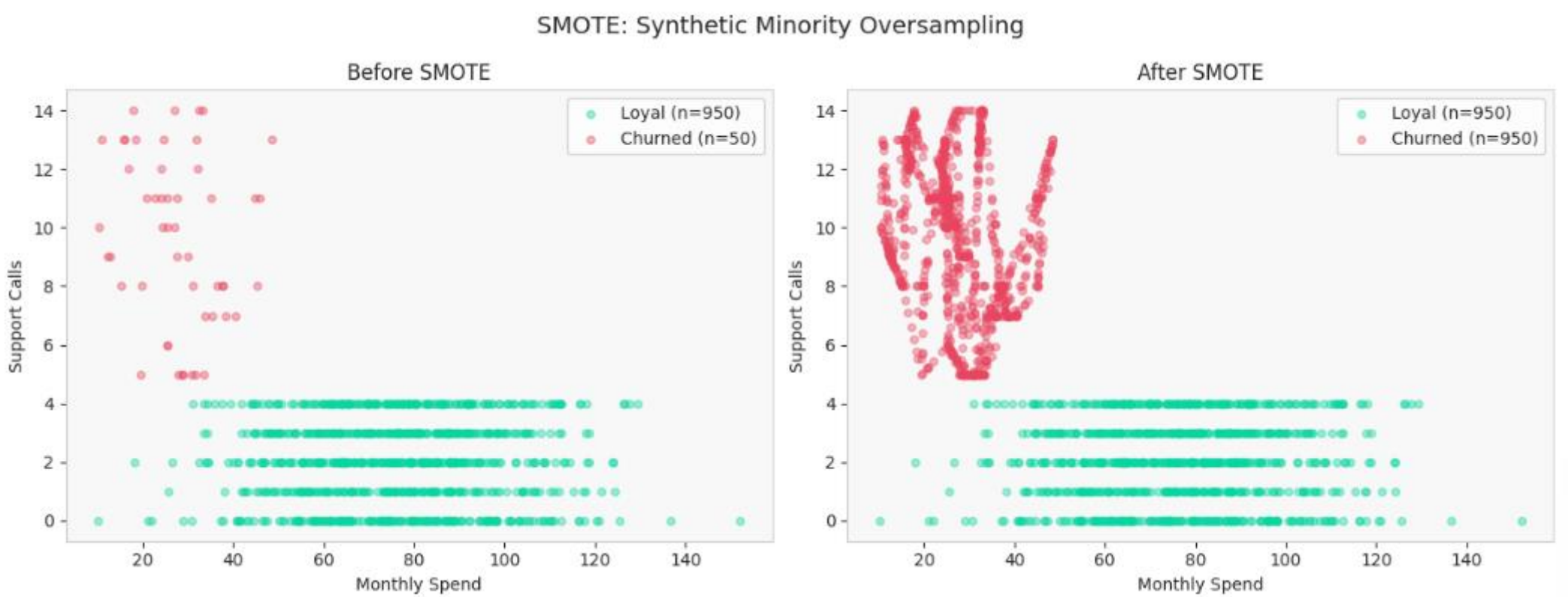
```
zip([0, 1], [700, 300])
# → (0, 700), (1, 300)
```

`dict(...)` Converts those pairs into a dictionary:

```
dict([(0, 700), (1, 300)])
# → {0: 700, 1: 300}
```

Final output:

`Before resampling: {0: 700, 1: 300}`


SMOTE: Synthetic Minority Oversampling
Before SMOTE
After SMOTE
Loyal (n=950)
Churned (n=50)
Churned (n=950)
Support Calls
Monthly Spend
0
2
4
6
8
10
12
14
20
40
60
80
100
120
140

8. 🏗️ Building Reusable Cleaning Pipelines

> Data cleaning pipelines **automate repetitive tasks**, ensure consistency, and make your code reusable.
>
> **Pipeline output should always be:**
>
> 1. ✅ **Clean Data** — ready for analysis/modeling
> 2. 🚫 **Quarantine File** — rejected records for human review
> 3. 📋 **Cleaning Log** — what changed, why, how many records affected

```
class DataCleaningPipeline:
    """
    Flexible, reusable data cleaning pipeline with configurable steps.

    Features:
    - Method chaining (add_step().add_step()...)
    - Automatic execution logging
    - Execution report generation
    """
```

```
def __init__(self, name="Data Cleaning Pipeline"):
    self.name = name
    self.steps = []
    self.execution_log = []

def add_step(self, name, function, **kwargs):
    """
    Add a cleaning step.

    Parameters
    ----------
    name     : str       – human-readable step name (appears in logs)
    function : callable  – function(df, **kwargs) → df
    **kwargs             – extra arguments passed to the function

    Returns self (enables method chaining)
    """
    self.steps.append({'name': name, 'function': function, 'kwargs': kwargs})
    return self  # ← enables method chaining: .add_step().add_step()
```

```python
def execute(self, df, verbose=True):
    """Run all steps in order and return cleaned DataFrame."""
    result = df.copy()  # ← Golden Rule #1
    self.execution_log = []

    for step in self.steps:
        if verbose:
            print(f"  ⚙  {step['name']}...")
        try:
            # Record before state
            rows_before = len(result)
            nulls_before = result.isna().sum().sum()
            # First .sum() Sums down each column (True=1, False=0)
            # Second .sum() Sums those column totals into a single number

            # Execute step
            result = step['function'](result, **step['kwargs'])

            # Record after state
            rows_after = len(result)
            nulls_after = result.isna().sum().sum()
```

```
            # Log execution
            self.execution_log.append(
                {
                    'step': step['name'],
                    'rows_before': rows_before,
                    'rows_after': rows_after,
                    'rows_changed': rows_after - rows_before,
                    'nulls_before': nulls_before,
                    'nulls_after': nulls_after,
                    'status': 'success',
                }
            )
        except Exception as e:
            self.execution_log.append(
                {'step': step['name'], 'status': 'failed', 'error': str(e)}
            )
            if verbose:
                print(f"    ❌ Error: {e}")
            raise

    if verbose:
        print(f"\n✅ Pipeline '{self.name}' completed successfully!")
    return result
```

```
    def get_execution_report(self):
        """Return a DataFrame summarizing each step's effect."""
        return pd.DataFrame(self.execution_log)

    def save_pipeline(self, filename):
        """
        Save pipeline configuration.
        """
        import pickle

        with open(filename, 'wb') as f:
            pickle.dump(self.steps, f)
        print(f"Pipeline saved to {filename}")

    def load_pipeline(self, filename):
        """
        Load pipeline configuration.
        """
        import pickle

        with open(filename, 'rb') as f:
            self.steps = pickle.load(f)
        print(f"Pipeline loaded from {filename}")
        return self


print("✅ DataCleaningPipeline class defined!")
```

`cat_cols = df.select_dtypes(include='object').columns`

`select_dtypes(include='object')` filters columns by their data type — `'object'` is how pandas stores **string/text columns**.

```
df = pd.DataFrame({
    'name':    ['Alice', 'Bob'],    # object  ✓
    'city':    ['Cairo', 'Paris'],  # object  ✓
    'age':     [25, 30],            # int64   ✗
    'salary':  [50.0, 60.0],        # float64 ✗
})

cat_cols = df.select_dtypes(include='object').columns
# Index(['name', 'city'], dtype='object')
```

```
# pandas 3 — more precise
cat_cols = df.select_dtypes(include='string').columns

# or catch both to be safe during transition
cat_cols = df.select_dtypes(include=['object', 'string']).columns
```

`.columns` at the end extracts just the column names as an Index (instead of returning the full DataFrame).

Common dtype filters:

```
df.select_dtypes(include='string')   # text columns (pandas 3 preferred)
df.select_dtypes(include='object')   # text columns (older style)
df.select_dtypes(include='number')   # all numeric (int + float)
df.select_dtypes(include='float')    # floats only
df.select_dtypes(include='int')      # integers only
df.select_dtypes(exclude='object')   # everything except text
```

```python
# — Define modular cleaning functions ——————————————————————————

def standardize_column_names(df):
    """Standardize column names. Lowercase + underscore column names."""
    df.columns = (
        df.columns.str.lower()
        .str.replace(' ', '_')
        .str.replace(r'[^a-z0-9_]', '', regex=True)
    )
    return df


def convert_data_types(df, type_mapping=None):
    """
    (Type Coercion) Convert data types based on mapping.
    """
    if type_mapping:
        for col, dtype in type_mapping.items():
            if col in df.columns:
                try:
                    df[col] = df[col].astype(dtype)
                except:
                    print(f"Warning: Could not convert {col} to {dtype}")
    return df
```

```python
def remove_duplicates(df, subset=None, keep='first'):
    """Drop duplicate rows."""
    return df.drop_duplicates(subset=subset, keep=keep)


def strip_string_cols(df):
    """Strip whitespace from all string columns."""
    cat_cols = df.select_dtypes(include=['object', 'string']).columns
    for col in cat_cols:
        df[col] = df[col].str.strip()
    return df


def handle_missing_numeric(df, strategy='median'):
    """Fill numeric NaNs with mean or median."""
    num_cols = df.select_dtypes(include=['number']).columns
    for col in num_cols:
        fill_val = df[col].mean() if strategy == 'mean' else df[col].median()
        df[col] = df[col].fillna(fill_val)
    return df


def handle_missing_categorical(df, fill_value='Unknown'):
    """Fill string NaNs with a constant."""
    cat_cols = df.select_dtypes(include='object').columns
    for col in cat_cols:
        df[col] = df[col].fillna(fill_value)
    return df
```

```python
def handle_missing_values(df, strategy='drop', threshold=0.5):
    """Handle missing values."""
    if strategy == 'drop':
        # Drop columns with too many missing values
        missing_pct = df.isna().sum() / len(df)
        cols_to_drop = missing_pct[missing_pct > threshold].index
        df = df.drop(columns=cols_to_drop)
    elif strategy == 'fill_median':
        numeric_cols = df.select_dtypes(include=['int64', 'float64']).columns
        for col in numeric_cols:
            df[col] = df[col].fillna(df[col].median())
    elif strategy == 'fill_mode':
        object_cols = df.select_dtypes(include='object').columns
        for col in object_cols:
            df[col] = df[col].fillna(
                df[col].mode()[0] if not df[col].mode().empty else 'Unknown'
            )
    return df


print("✅ Cleaning functions defined!")
```

```python
# ── Build and execute the pipeline ──────────────────────────────────────
pipeline = DataCleaningPipeline("Customer Data Cleaning")

(
    pipeline.add_step("Standardize Column Names", standardize_column_names)
    .add_step("Strip String Whitespace", strip_string_cols)
    .add_step("Remove Duplicates", remove_duplicates, subset=['customer_id'])
    .add_step("Handle Missing Numeric", handle_missing_numeric, strategy='median')
    .add_step(
        "Handle Missing Categorical", handle_missing_categorical, fill_value='Unknown'
    )
)
# NOTE: The outer () is just a Python trick to allow line continuation without needing backslashes.
```

```python
df_cleaned = pipeline.execute(df_messy)
```

⚙️ Standardize Column Names...
⚙️ Strip String Whitespace...
⚙️ Remove Duplicates...
⚙️ Handle Missing Numeric...
⚙️ Handle Missing Categorical...

✅ Pipeline 'Customer Data Cleaning' completed successfully!

UNcleaned Data:

	Customer ID	Customer Name	Age	Purchase Amt	Category
0	1	Alice	25.0	100.0	Electronics
1	2	BOB	NaN	250.5	NaN
2	2	BOB	30.0	250.5	Electronics
3	3	Charlie	35.0	NaN	Clothing
4	4	NaN	40.0	400.0	Books
5	5	Eve	NaN	550.0	NaN
6	6	Frank	28.0	75.0	Clothing

📊 Cleaned Data:

	customer_id	customer_name	age	purchase_amt	category
0	1	Alice	25.0	100.0	Electronics
1	2	BOB	31.5	250.5	Unknown
3	3	Charlie	35.0	250.5	Clothing
4	4	Unknown	40.0	400.0	Books
5	5	Eve	31.5	550.0	Unknown
6	6	Frank	28.0	75.0	Clothing

Shape: (6, 5) | Missing values: 0

```python
# ── Execution Report ──────────────────────────────────────────────
report = pipeline.get_execution_report()
print("📋 Execution Report:")
print(
    report[
        [
            'step',
            'rows_before',
            'rows_after',
            'rows_changed',
            'nulls_before',
            'nulls_after',
            'status',
        ]
    ].to_string(index=False)
)
```

```
📋 Execution Report:
                    step  rows_before  rows_after  rows_changed  nulls_before  nulls_after  status
  Standardize Column Names            7           7             0             6            6 success
   Strip String Whitespace            7           7             0             6            6 success
         Remove Duplicates            7           6            -1             6            6 success
    Handle Missing Numeric            6           6             0             6            3 success
Handle Missing Categorical            6           6             0             3            0 success
```

📖 Summary: Complete Data Cleaning Cheat Sheet

Issue	Detection	Strategy	Key Functions
Accuracy	Domain rules	Rule-based fix or reject	`apply()`, `loc[]`, boolean masks
Consistency	`value_counts()`	Standardize + type coerce	`str.lower()`, `.map()`, `pd.to_numeric()`
Missing (MCAR<5%)	`isnull().sum()`	Delete rows	`dropna()`
Missing (numeric)	`isnull()` heatmap	Mean / Median imputation	`SimpleImputer`, `fillna()`
Missing (categorical)	`isnull()`	Mode or 'Unknown'	`fillna('Unknown')`
Missing (advanced)	Correlation check	KNN / MICE	